Exploring application of LLM on Games

Abstract

There are many different applications of Large Language Models (LLMs for short) nowadays, this report focuses on an attempt of using LLMs to play a classical Chinese poker game, named Doudizhu (We will call it “Chinese Poker” in this report). To this end, we will first compare different approaches of using AI programs to play the game and indentify the advantages of ultilizing LLMs. Furthermore, the report will introduce the computer gaming program and the settings of the AI, explaining how the game data are transferred between programs, and how the LLM will react based on the information provided. Finally, we will also introduce the future plan of this project.

Background

Games are a classical topic in AI, and there are many ways to train a model or design a program to play games. The popular approach is Minimax, which is a recursive algorithm to compute the future steps of the game and find the optimal step for the player and the opponent. This algorithm works by computing all the possible steps and choosing the best one, or using probability theory to find the step that is most likely to win. But this approach is only suitable for perfect-information games. In the game we are focusing on, Chinese Poker is an imperfect-information game, where the players cannot see each other's cards, and the computer program cannot find the best step based only on the player's cards.

Researchers have been using the minimax algorithm to play imperfect-information games such as Texas hold'em poker. By incorporating chance nodes, the algorithm can estimate the probability of winning in a given unknown scenario. This implementation requires extensive calculations on probabilities, and the complexity increases as the game becomes more complex.

Machine learning can also be used to play games. By feeding the AI enough data to train a machine learning model, the AI can play the game using the “knowledge” it gained from the training process. This approach involves collecting a large dataset of gameplay examples, including the game state, actions taken, and outcomes. This data is then used to train a machine learning model, such as a neural network, to learn patterns and correlations between game states and optimal actions. With the trained model, the AI can then play the game autonomously, using the knowledge it gained from the training data to make decisions and win the game. This approach has led to remarkable achievements, such as AI systems beating the world's best human players in complex games like Go. The success of this approach depends on having access to high-quality training data.

Deep reinforcement learning is also a sub-topic of machine learning. In this approach, the training data does not consist of the rules of the actual game. Instead, the reinforcement learning program plays the game and learns from the process. The key feature of reinforcement learning is that it allows the program to discover a strategy that maximizes the score by setting appropriate rewards.

There are several examples of reinforcement learning being applied to popular games, such as Snake, Tetris, and even racing games. With sufficient training time and computational resources, the model can be optimized for playing these games.

Advantages of using LLMs to Play Games

Besides Minimax and deep learning, we chose to use an LLM to play the game. As we mentioned before, it is difficult to apply Minimax to an imperfect-information game, so we should find a better choice.

Reinforcement learning can be a choice for playing the game, but there are several drawbacks to this approach. First, it will consume a huge amount of time for training the model, and this process also requires great computational power. Second, since we train the model by making

it play the game, the AI does not know how to actually play the game, so it is not possible to make it provide explanations about the steps it plays. Moreover, if there is not enough data about the game, it is difficult to let the AI player respond as a human player.

By our common sense, humans learn how to play a game from information from other players and practicing the game. For ML algorithms, they learn fully from practice, which requires a lot of practice. But LLM models have knowledge from humans; they learn from the information on the internet, which can give them a better understanding of the game, and it is also possible for an LLM model to provide explanations of their steps.

This report will focus on the last method we mentioned. By implementing a game program and making an LLM play the game, it can show that LLMs can also be a possible way to play popular games such as Chinese Poker.

The Advantage of ChatGLM

Selecting a proper LLM model will be essential to the performance of the AI. There are many options we can choose, but we will choose ChatGLM-6B for this report.

ChatGLM is an open bilingual language model based on the GLM (General Language Model) framework. It is optimized for Chinese dialogue, so it can work better for the Chinese Poker game. Although this report hasn't tried other LLM models, we believe ChatGLM-6B is capable for this task.

ChatGLM is easy to implement and uses a small amount of computational resources to apply. In the next steps of this research, it will be easier to fine-tune the model, although more proof should be discussed in the future.

The game setting

The game program used Python, which makes it easier to implement the transformer libraries of Python to get the AI to respond.

To have better programming organization, there are different classes and functions created in the game.

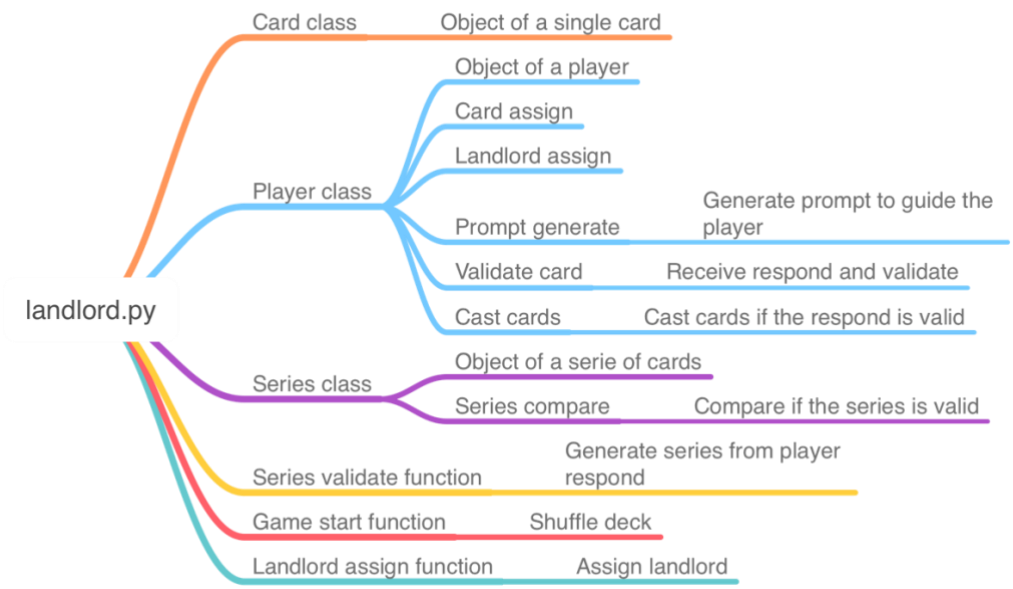

Figure1: The structure of the game program

Card class: Chinese poker uses a standard poker deck with both black & white and colored jokers. For easier comparison of the cards, the card class contains basic information of a card, including suit, rank, and a unique ID to distinguish the cards.

```
class Card:
    # Constructor: input two integer: suit and value
    def __init__(self, suit, value):
        self.suit = suit
        self.value = value
        self.id = 4 * value + suit
```

Player class: The Player class contains many different methods. The most important part is casting cards. The methods can take a string as input, which contains the cards the player is going to play, and determine if it is possible to cast the cards by calling a series validation function. If it is possible, the function can delete the cards that are going to be played from the player's cards.

```
    def castCards(self, cast):
        for i in range (len(cast)):
```

```
            self.cards.remove(cast[i])
    def castCards(self, cast):
        for i in range (len(cast)):
            self.cards.remove(cast[i])
```

The class also contains basic getter and setter methods, besides some necessary interaction functions between the game program and the player.

```
class player:
    def __init__(self, name):
        self.name = name
        self.landLord =  False
```

Series class:

A series is the set of cards that the player has played. Instead of using a list of cards to show the cards the player is going to play, the class contains essential information about the series, including the number of cards in the series and the value of the series.

```
class series:
    # constructor, create a serires object by the serires's type, value, amount of cards(连对, 顺子), addons(三带, 飞机, 四带)
    def __init__(self, seriesCards = [], type = '违规', value = 0, amount = 0, addOn1 = 0, addOn2 = 0):
        self.seriesCards = seriesCards
        self.type = type
        self.value = value
        self.amount = amount
        self.addOn1 = addOn1
        self.addOn2 = addOn2
```

The compare method can compare the series to other series.

```
    def compare(self, lower):
        if lower.type == '违规' or self.type == '违规':
            return False
        if lower.type == '王炸':
            return True
        if self.type == '王炸':
            return False
        if lower.type == '炸弹' and self.type != '炸弹':
            return True
        if lower.type != '炸弹' and self.type == '炸弹':
            return False
```

```
        if lower.type != self.type or lower.amount !=
self.amount or lower.addOn1 != self.addOn1 or lower.addOn2 !=
self.addOn2:
            return False
        return lower.value > self.value
```

Series validation function:

The series validation function can take a list of cards which the player has played, and construct a series object to describe it. If the cards cannot form a legal series, the function can return false to let the player play a list of legal cards.

```
def seriesValidate(cards):
    values = [card.value for card in cards]
    values.sort()

    length = len(cards)

    if length == 0:
        # 空
        return series(seriesCards = cards)

    if length == 1:
        # 单牌
        return series(seriesCards = cards, type = '单牌', value
= values[0])

    if length == 2:
        # 对子 或 王炸
        if values[0] == values[1]:
            return series(seriesCards = cards, type = '对子',
value = values[0])
        if values[0] == 13 and values[1] == 14:
            return series(seriesCards = cards, type = '王炸')

    if length == 3:
        # 三
        if values[0] == values[1] and values[1] == values[2]:
            return series(seriesCards = cards, type = '三带',
value = values[0])

    if length == 4:
        # 三带一 或 炸弹
```

```
        if (values[0] == values[1] and values[1] == values[2]
and values[2] != values[3]
            or values[1] == values[2] and values[2] ==
values[3] and values[3] != values[0]
            ):
            return series(seriesCards = cards, type = '三带',
value = values[1], addOn1 = 1)
        if values[0] == values[1] and values[1] == values[2]
and values[2] == values[3]:
            return series(seriesCards = cards, type = '炸弹',
value = values[0])

    if length == 5:
        # 三带一对
        if (values[0] == values[1] and values[1] == values[2]
and values[2] != values[3] and values[3] == values[4]
            or values[2] == values[3] and values[3] ==
values[4] and values[4] != values[0] and values[0] ==
values[1]
            ):
            return series(seriesCards = cards, type = '三带',
value = values[2], amount = length)

    # 顺子，连对 不能以 2 结尾
    if values[length-1] == 12:
        return series(seriesCards = cards)

    # 顺子
    if length > 4:
        straight = True
    else:
        straight = False

    for i in range(length):
        if i > 0 and values[i] != values[i-1] + 1:
            straight = False
    if straight:
            return series(seriesCards = cards, type = '顺子',
value = values[length-1], amount = length)

    # 连对
    if length > 5 and length % 2 == 0:
        straightPairs = True
    else:
        straightPairs = False
    for i in range(length):
        if i  == 0:
```

```
            continue
        if i % 2 == 1 and values[i] != values[i - 1]:
            straightPairs = False
        if i % 2 == 0 and values[i] != values[i - 2] + 1:
            straightPairs = False

    if straightPairs:
            return series(seriesCards = cards, type = '连对',
value = values[length-1], amount = length)

    # 错误牌型
    return series(seriesCards = cards)
```

Game start function:

The game start function randomly assigns cards to the players. And it can output information about the game, so the players can react based on the game.

```
def gameStart(players):
    deck = [Card(suit, value) for suit in range(4) for value
in range(13)]
    deck.append(Card(4, 13))
    deck.append(Card(4, 14))

    random.shuffle(deck)

    players[0].cardsAssign(deck[0:17])
    players[1].cardsAssign(deck[17:34])
    players[2].cardsAssign(deck[34:51])

    return deck[51:54]
```

Landlord decision function:

There is a special player called the landlord in the Chinese poker game. The landlord decision function will be called after the game start function. It can either randomly or manually assign the landlord.

```
def landlordDecide(players, landLordCards, landLordNumber = 0,
rand = True):
    if rand:
        landLordNumber = random.randrange(3)

    players[landLordNumber].becomeLandLord(landLordCards)

    return landLordNumber, f"{players[0]}" + f"{players[1]}" +
```

```
f"{players[2]}" + "地主牌: " + f"{landLordCards[0]}" +
f"{landLordCards[1]}" + f"{landLordCards[2]}"
```

How the LLM reacts

There are many possible ways to get the LLM to respond. Since there are histories of the LLM dialogue, we can let the LLM player react from the chat history and the game information. But doing this consumes more computational resources, since Chinese Poker involves many turns and there is a lot of information in each turn, such as the player's cards, the number of cards of other players, cards on the table, and the remaining cards. This information contributes to a huge consumption of computational power.

There are other ways to optimize the game. Since there is no need to focus on the past turns of the game, the LLM can react only to the cards left in the game and the player's cards. Thus, by making prompts, we can provide the necessary information to the LLM, and save computational power.

Fine tuning LLM

For better performance of the language model, there are ways to improve. The best way is fine-tuning the model.

Like other LLMs, we can use information about the Chinese Poker game, such as blogs, articles, and online discussions about the game. This is the most common way to improve the game, because it can let the model gain more knowledge about the game. It can provide more information about each move of the player, and explain it in a human-like or the most popular way among the players.

The other method is to use reinforcement learning. By setting rewards for the game, the model can learn the most winning way to play. But this method needs more time to train, because the model needs to play a great number of rounds of the game.

In the next part of this research, we will comapre the advantages of both methods.

Further steps:

As mentioned in our report, there are many future directions to get a better understanding of LLM play in Chinese poker.

We tried ChatGLM-6B in this research, but there are many other models that have bilingual capability. We can use the game setup to find which model has the best ability to play the game. Besides using existing models, we can fine-tune those models to have better performance. As mentioned in the fine-tuning part, we can try both methods or combine them together, then test them. By making them play the same game, we can find which way can be more effective.

Future applications:

Besides playing the Chinese poker game, there are many games that LLMs can play, especially card games, since they do not need rapid reaction time. Or even some popular PC games, and there are already people who have applied LLMs to them.

For applications, LLM playing games has one significant benefit: it can tell the player why they are playing the cards in a certain way. It can act as a tutor to teach the player how to play the game, or play with real people.

Reference

Brown, N., & Sandholm, T. (2018). Superhuman AI for heads-up no-limit poker: Libratus beats top professionals. *Science*, *359*(6374), 418–424. https://doi.org/10.1126/science.aao1733

Comi, M. (2022, June 8). How to teach AI to play Games: Deep Reinforcement Learning. *Medium*. https://towardsdatascience.com/how-to-teach-an-ai-to-play-games-deep-reinforcement-learning-28f9b920440a

L, R. (2022, January 6). Reinforcement Learning on Tetris - Rex L - Medium. *Medium*. https://rex-l.medium.com/reinforcement-learning-on-tetris-707f75716c37

Luchang. (n.d.). *GitHub - luchang1113/HEP-LLM-play-StarCraftII*. GitHub. https://github.com/luchang1113/HEP-LLM-play-StarCraftII

Thudm. (n.d.). *GitHub - THUDM/ChatGLM-6B: ChatGLM-6B: An Open Bilingual Dialogue Language Model | 开源双语对话语言模型*. GitHub. https://github.com/THUDM/ChatGLM-6B

Worktile. (2024, May 16). *斗地主详细编程代码是什么*. Worktile 社区. https://worktile.com/kb/ask/2194245.html